❖ Definition of the Algorithm

The Minimum Edit Distance (MED) algorithm calculates the minimum number of operations required to transform one word into another. The supported operations include:

- **Insertion:** Adding a character to the string.
- **Deletion:** Removing a character from the string.
- **Substitution:** Replacing one character with another.

This implementation is split into functions and procedures to handle different functionalities, such as GUI design, calculating MED, reading a vocabulary file, and finding the closest words.

1. Main Input and Output

- **Input:**

  User Inputs (via GUI):

  Word(s) to calculate MED.

  Selection of the functionality (Part 1: Closest Words, Part 2: Transformation Steps).

  File Input:

  Vocabulary file (vocabulary_tr.txt).

- **Output:**

  For Part 1:

  Displays the five closest words to the input word along with their MED values.

  For Part 2:

  Displays the step-by-step transformation from word1 to word2.

  Outputs the minimum edit distance.

2. Functions and Procedures

```
public static int minDistance(String word1,String word2,int opNumber,StringBuilder steps) {  //The function for the MED algorithm

    final int length1= word1.length(); //The length of the word1
    final int length2= word2.length(); //The length of the word2

    int [][] minOp=new int[length2+1][length1+1];  //Initializing the cost matrix


    minOp[0][0]=0; //Filling the left top corner of minOp matrix with 0

    for(int i=1;i<=length1;i++) { //Filling the first row
        minOp[0][i]=i;
    }

    for(int j=1;j<=length2;j++) { //Filling the first column
        minOp[j][0]=j;
    }
```

'minDistance' 1

`minDistance(String word1, String word2, int opNumber, StringBuilder steps)`

- **Purpose:**

- Calculates the minimum number of operations to transform word1 into word2 using a dynamic programming approach.

- **Inputs:**

- word1: The first word (source string).

-word2: The second word (target string).

-opNumber: Determines whether to calculate only the MED (1) or to also generate transformation steps (2).

-steps: A StringBuilder to store the transformation steps when opNumber is 2.

- **Outputs:**

- Returns the minimum edit distance (an integer).

-If opNumber == 2, appends the transformation steps to the steps parameter.

- **Procedure:**

1. Initialization: Creates a 2D cost matrix minOp with dimensions [length2 + 1][length1 + 1] to store intermediate results.
2. Base Cases: Initializes the first row and column of the matrix with incremental values.
3. Filling the Matrix: Iteratively calculates the cost for each position based on the previous values (top-left, top, and left cells).
4. Backtracking (if opNumber == 2): Determines the transformation steps by backtracking through the matrix.

```java
public static List<String> readFile(String word) throws IOException {  //The function to read the file
    FileReader fReader= new FileReader("vocabulary_tr.txt");
    String line;
    BufferedReader brReader= new BufferedReader(fReader);

    List<String> vocabulary = new ArrayList<>(); //The list for words in file

    while((line=brReader.readLine()) != null ) { //Getting the rows of file
        vocabulary.add(line.trim());    //Add the words to the list
    }

    return vocabulary;
}
```

` readFile(String word)` 2

` readFile(String word)`

- **Purpose:**

- Reads a vocabulary file (vocabulary_tr.txt) line by line and returns a list of words.

- **Inputs:**

- word: Placeholder input, not directly used in the function.

- **Outputs:**

- Returns a List<String> containing all words from the file.

- **Procedure:**

1. Opens the file using a FileReader.
2. Reads each line and trims any whitespace.
3. Adds each word to a list and returns the list.

```java
//The function to apply MED and find the five closest words
public static void findFiveClosest(String word, List <String> vocabulary,StringBuilder result){

    HashMap<String,Integer> minDistanceMap= new HashMap<String,Integer>();

    for(String vocabWord:vocabulary) {
        int numOfOp= minDistance(word, vocabWord, 1,result); //Calculating the number of operations with the minDistance
        minDistanceMap.put(vocabWord,numOfOp);  //Putting the words and its number of operations to the map
    }

    List<Map.Entry<String, Integer>> entryList = new ArrayList<>(minDistanceMap.entrySet());

    //Sorting according to the values
    entryList.sort(Map.Entry.comparingByValue());

    int count = 0; //Counter to print the 5 closest words

    //Print the sorted entries
    for (Map.Entry<String, Integer> entry : entryList) {

        result.append(entry.getKey()).append(": ").append(entry.getValue()).append("\n");
        count++;

        if (count == 5) { // Break the loop after printing 5 element
            break;
        }

    }
}
```

` *findFiveClosest(String word, List<String> vocabulary, StringBuilder result)* ` *3*

` findFiveClosest(String word, List<String> vocabulary, StringBuilder result)`

- **Purpose:**

  - Finds the five closest words in the vocabulary to the input word based on MED.

- **Inputs:**

  - word: The input word.

  -vocabulary: A list of words to compare against.

  -result: A StringBuilder to store the five closest words and their distances.

- **Outputs:**

  - Updates the result with the closest five words and their respective MED values.

- **Procedure:**

  1. Calculates the MED for each word in the vocabulary using minDistance.
  2. Stores the words and their MED values in a HashMap.
  3. Sorts the map by values in ascending order.
  4. Appends the top five entries to the result.

```java
public static void createAndShowGUI() {  //The function for ui design
    // Main frame
    JFrame frame = new JFrame("Minimum Edit Distance Tool");
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.setSize(600, 400);

    //Main pannel
    JPanel mainPanel = new JPanel(new BorderLayout(10, 10));

    // Header
    JLabel titleLabel = new JLabel("Minimum Edit Distance Tool", SwingConstants.CENTER);
    titleLabel.setFont(new Font("Arial", Font.BOLD, 20));
    mainPanel.add(titleLabel, BorderLayout.NORTH);

    // Enterence pannel
    JPanel inputPanel = new JPanel(new GridLayout(4, 2, 10, 10));

    JLabel partLabel = new JLabel("Select Part:");
    JComboBox<String> partSelector = new JComboBox<>(new String[]{"Part 1", "Part 2"});
    inputPanel.add(partLabel);
    inputPanel.add(partSelector);
```

` createAndShowGUI()` 4

` createAndShowGUI()`

- **Purpose:**

- Designs and initializes the graphical user interface (GUI).

- **Inputs:**

- None (called directly within the main method).

- **Procedure:**

1. Creates a JFrame and organizes components using JPanel with BorderLayout and GridLayout.

2. Includes input fields, dropdowns, buttons, and a result display area.

3. Adds action listeners for user interactions, such as calculating MED or finding closest words.

- **Outputs:**

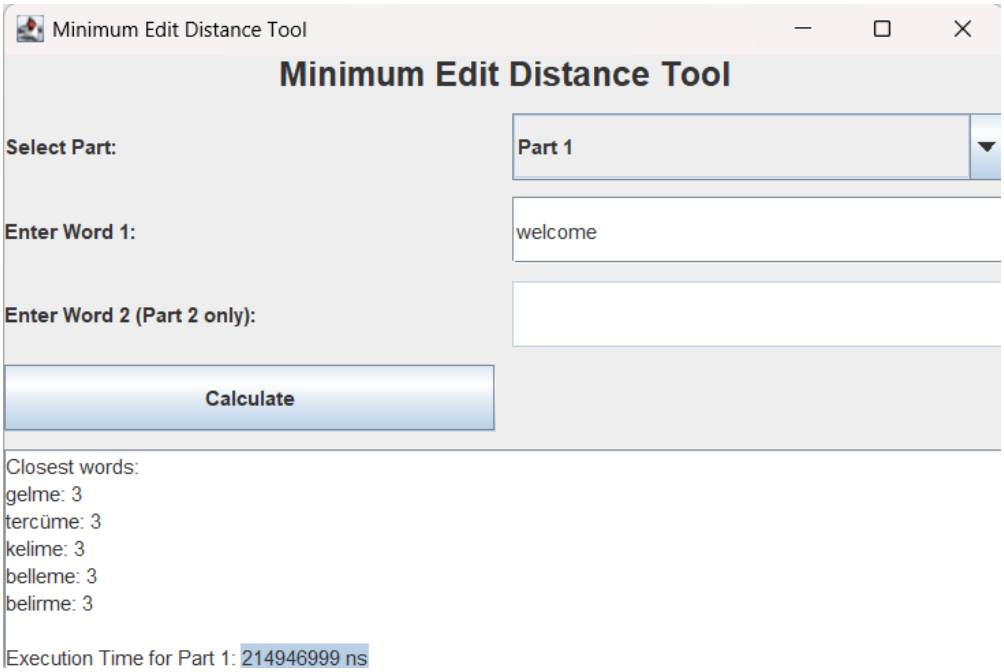
- Displays the GUI window.

5. Test Results

Part1:

Input: “welcome”

Total Running Time: 214946999 ns



Output:

Closest words:

gelme: 3

tercüme: 3

kelime: 3

belleme: 3

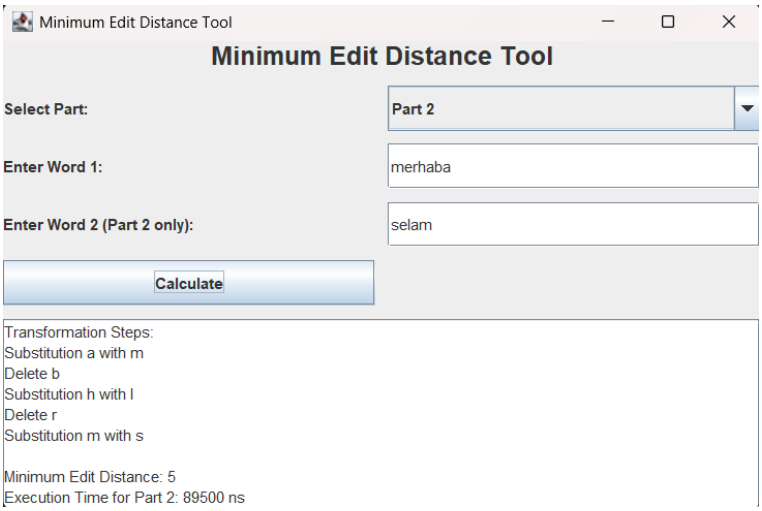
belirme: 3

Part2:

Input: “merhaba”

Input2: “selam”

Total Running Time: 89500 ns



Output:

Transformation Steps:

Substitution a with m

Delete b

Substitution h with l

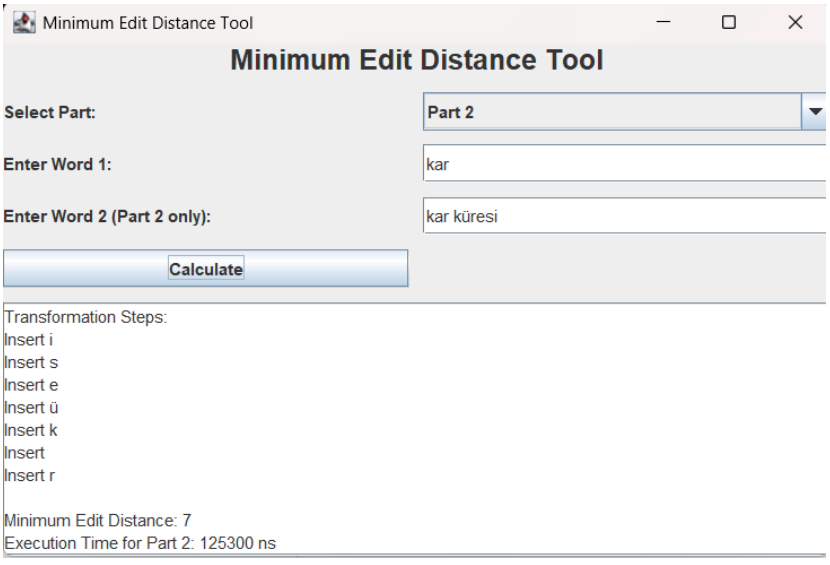
Delete r

Substitution m with s

Input: "merhaba"

Input2: "selam"

Total Running Time: 125300 ns



Output:

Transformation Steps:

Insert i

Insert s

Insert e

Insert ü

Insert k

Insert

Insert r